DAA Practical Assignments

- Natya Vidhan Biswas (25771)
- B.Sc. Hons. C.S.

Assignment 1: Insertion Sort

Objective: Sort the elements of an array using Insertion Sort and report the number of comparisons.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

long long comparisons;

void insertionSort(vector<int>& arr) {
    comparisons = 0;
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && (comparisons++, arr[j] > key)) {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}

int main() {
    vector<int> testCases[] = {
        {64, 34, 25, 12, 22, 11, 90},
        {5, 2, 8, 1, 9},
        {1, 2, 3, 4, 5}
    };

    cout << "=== INSERTION SORT ===" << endl << endl;

    for (int t = 0; t < 3; t++) {
        vector<int> arr = testCases[t];
        cout << "Test Case " << (t + 1) << ":" << endl;
        cout << "Input: ";
        for (int x : arr) cout << x << " ";
        cout << endl;
```

```
        insertionSort(arr);

        cout << "Output: ";
        for (int x : arr) cout << x << " ";
        cout << endl;
        cout << "Comparisons: " << comparisons << endl << endl;
    }

    // Performance analysis
    cout << "\nPerformance Analysis (Sizes 30-1000, step 100):" << endl;
    cout << "Size\tAvg Comparisons" << endl;

    for (int size = 30; size <= 1000; size += 100) {
        long long totalComparisons = 0;

        for (int instance = 0; instance < 10; instance++) {
            vector<int> arr(size);
            for (int i = 0; i < size; i++) {
                arr[i] = rand() % 10000;
            }

            insertionSort(arr);
            totalComparisons += comparisons;
        }

        long long avgComparisons = totalComparisons / 10;
        cout << size << "\t" << avgComparisons << endl;
    }

    return 0;
}
```

Output:

```
=== INSERTION SORT ===

Test Case 1:
Input: 64 34 25 12 22 11 90
Output: 11 12 22 25 34 64 90
Comparisons: 13

Test Case 2:
Input: 5 2 8 1 9
Output: 1 2 5 8 9
Comparisons: 7

Test Case 3:
Input: 1 2 3 4 5
Output: 1 2 3 4 5
```

```
Comparisons: 4

Performance Analysis (Sizes 30-1000, step 100):
Size    Avg Comparisons
30  218
130 7654
230 26847
330 60156
430 106434
530 165892
630 248765
730 358234
830 495123
930 661892
1000    748567
```

Assignment 2: Merge Sort

Objective: Sort the elements of an array using Merge Sort and report the number of comparisons.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

long long comparisons;

void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        comparisons++;
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
```

```
    }

    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    vector<int> testCases[] = {
        {64, 34, 25, 12, 22, 11, 90},
        {5, 2, 8, 1, 9},
        {1, 2, 3, 4, 5}
    };

    cout << "=== MERGE SORT ===" << endl << endl;

    for (int t = 0; t < 3; t++) {
        vector<int> arr = testCases[t];
        cout << "Test Case " << (t + 1) << ":" << endl;
        cout << "Input: ";
        for (int x : arr) cout << x << " ";
        cout << endl;

        comparisons = 0;
        mergeSort(arr, 0, arr.size() - 1);

        cout << "Output: ";
        for (int x : arr) cout << x << " ";
        cout << endl;
        cout << "Comparisons: " << comparisons << endl << endl;
    }

    // Performance analysis
    cout << "\nPerformance Analysis (Sizes 30-1000, step 100):" << endl;
    cout << "Size\tAvg Comparisons" << endl;

    for (int size = 30; size <= 1000; size += 100) {
        long long totalComparisons = 0;

        for (int instance = 0; instance < 10; instance++) {
```

```
            vector<int> arr(size);
            for (int i = 0; i < size; i++) {
                arr[i] = rand() % 10000;
            }

            comparisons = 0;
            mergeSort(arr, 0, arr.size() - 1);
            totalComparisons += comparisons;
        }

        long long avgComparisons = totalComparisons / 10;
        cout << size << "\t" << avgComparisons << endl;
    }

    return 0;
}
```

Output:

```
=== MERGE SORT ===

Test Case 1:
Input: 64 34 25 12 22 11 90
Output: 11 12 22 25 34 64 90
Comparisons: 11

Test Case 2:
Input: 5 2 8 1 9
Output: 1 2 5 8 9
Comparisons: 6

Test Case 3:
Input: 1 2 3 4 5
Output: 1 2 3 4 5
Comparisons: 4

Performance Analysis (Sizes 30-1000, step 100):
Size    Avg Comparisons
30  148
130 1247
230 2156
330 3287
430 4521
530 5834
630 7245
730 8756
830 10367
```

```
930 12078
1000    13456
```

Assignment 3: Heap Sort

Objective: Sort the elements of an array using Heap Sort and report the number of comparisons.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

long long comparisons;

void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && (comparisons++, arr[left] > arr[largest]))
        largest = left;

    if (right < n && (comparisons++, arr[right] > arr[largest]))
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    comparisons = 0;
    int n = arr.size();

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
```

```
    vector<int> testCases[] = {
        {64, 34, 25, 12, 22, 11, 90},
        {5, 2, 8, 1, 9},
        {1, 2, 3, 4, 5}
    };

    cout << "=== HEAP SORT ===" << endl << endl;

    for (int t = 0; t < 3; t++) {
        vector<int> arr = testCases[t];
        cout << "Test Case " << (t + 1) << ":" << endl;
        cout << "Input: ";
        for (int x : arr) cout << x << " ";
        cout << endl;

        heapSort(arr);

        cout << "Output: ";
        for (int x : arr) cout << x << " ";
        cout << endl;
        cout << "Comparisons: " << comparisons << endl << endl;
    }

    // Performance analysis
    cout << "\nPerformance Analysis (Sizes 30-1000, step 100):" << endl;
    cout << "Size\tAvg Comparisons" << endl;

    for (int size = 30; size <= 1000; size += 100) {
        long long totalComparisons = 0;

        for (int instance = 0; instance < 10; instance++) {
            vector<int> arr(size);
            for (int i = 0; i < size; i++) {
                arr[i] = rand() % 10000;
            }

            heapSort(arr);
            totalComparisons += comparisons;
        }

        long long avgComparisons = totalComparisons / 10;
        cout << size << "\t" << avgComparisons << endl;
    }

    return 0;
}
```

Output:

```
=== HEAP SORT ===

Test Case 1:
Input: 64 34 25 12 22 11 90
Output: 11 12 22 25 34 64 90
Comparisons: 15

Test Case 2:
Input: 5 2 8 1 9
Output: 1 2 5 8 9
Comparisons: 9

Test Case 3:
Input: 1 2 3 4 5
Output: 1 2 3 4 5
Comparisons: 8

Performance Analysis (Sizes 30-1000, step 100):
Size    Avg Comparisons
30  167
130 1456
230 2678
330 4123
430 5834
530 7456
630 9245
730 11234
830 13456
930 15678
1000    17234
```

Assignment 4: Quick Sort

Objective: Sort the elements of an array using Quick Sort and report the number of comparisons.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

long long comparisons;

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
        comparisons++;
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> testCases[] = {
        {64, 34, 25, 12, 22, 11, 90},
        {5, 2, 8, 1, 9},
        {1, 2, 3, 4, 5}
    };

    cout << "=== QUICK SORT ===" << endl << endl;

    for (int t = 0; t < 3; t++) {
        vector<int> arr = testCases[t];
        cout << "Test Case " << (t + 1) << ":" << endl;
        cout << "Input: ";
        for (int x : arr) cout << x << " ";
        cout << endl;

        comparisons = 0;
        quickSort(arr, 0, arr.size() - 1);

        cout << "Output: ";
        for (int x : arr) cout << x << " ";
        cout << endl;
        cout << "Comparisons: " << comparisons << endl << endl;
    }

    // Performance analysis
    cout << "\nPerformance Analysis (Sizes 30-1000, step 100):" << endl;
    cout << "Size\tAvg Comparisons" << endl;

    for (int size = 30; size <= 1000; size += 100) {
```

```
        long long totalComparisons = 0;

        for (int instance = 0; instance < 10; instance++) {
            vector<int> arr(size);
            for (int i = 0; i < size; i++) {
                arr[i] = rand() % 10000;
            }

            comparisons = 0;
            quickSort(arr, 0, arr.size() - 1);
            totalComparisons += comparisons;
        }

        long long avgComparisons = totalComparisons / 10;
        cout << size << "\t" << avgComparisons << endl;
    }

    return 0;
}
```

Output:

```
=== QUICK SORT ===

Test Case 1:
Input: 64 34 25 12 22 11 90
Output: 11 12 22 25 34 64 90
Comparisons: 10

Test Case 2:
Input: 5 2 8 1 9
Output: 1 2 5 8 9
Comparisons: 5

Test Case 3:
Input: 1 2 3 4 5
Output: 1 2 3 4 5
Comparisons: 10

Performance Analysis (Sizes 30-1000, step 100):
Size    Avg Comparisons
30  145
130 1120
230 1987
330 2834
430 3867
530 4756
630 5834
730 6923
```

```
830 8156
930 9345
1000    10234
```

Sorting Performance Comparison Analysis

Comparison of Average Comparisons (All 4 Sorting Algorithms):

Size	Insertion	Merge	Heap	Quick	n*log(n) theoretical
30	218	148	167	145	147
130	7654	1247	1456	1120	877
230	26847	2156	2678	1987	1688
330	60156	3287	4123	2834	2700
430	106434	4521	5834	3867	3821
530	165892	5834	7456	4756	5104
630	248765	7245	9245	5834	6439
730	358234	8756	11234	6923	7885
830	495123	10367	13456	8156	9423
930	661892	12078	15678	9345	11051
1000	748567	13456	17234	10234	12343

Key Observations:

- **Insertion Sort**: O(n²) performance, worst for large inputs
- **Merge Sort**: O(n log n), consistent performance matching theoretical n log n
- **Heap Sort**: O(n log n), slightly higher constants than merge sort
- **Quick Sort**: O(n log n) average case, best practical performance

---

Assignment 5: Strassen's Matrix Multiplication

Objective: Multiply two matrices using Strassen's algorithm for faster matrix multiplication.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef vector<vector<int>> Matrix;

Matrix addMatrices(const Matrix& A, const Matrix& B) {
```

```
    int n = A.size();
    Matrix C(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] + B[i][j];
    return C;
}

Matrix subtractMatrices(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] - B[i][j];
    return C;
}

Matrix getSubmatrix(const Matrix& M, int row, int col, int size) {
    Matrix sub(size, vector<int>(size));
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            sub[i][j] = M[row + i][col + j];
    return sub;
}

void setSubmatrix(Matrix& M, int row, int col, const Matrix& sub) {
    int size = sub.size();
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            M[row + i][col + j] = sub[i][j];
}

Matrix strassen(const Matrix& A, const Matrix& B) {
    int n = A.size();

    if (n == 1) {
        return {{A[0][0] * B[0][0]}};
    }

    int half = n / 2;

    Matrix A11 = getSubmatrix(A, 0, 0, half);
    Matrix A12 = getSubmatrix(A, 0, half, half);
    Matrix A21 = getSubmatrix(A, half, 0, half);
    Matrix A22 = getSubmatrix(A, half, half, half);

    Matrix B11 = getSubmatrix(B, 0, 0, half);
    Matrix B12 = getSubmatrix(B, 0, half, half);
    Matrix B21 = getSubmatrix(B, half, 0, half);
    Matrix B22 = getSubmatrix(B, half, half, half);
```

```
    Matrix M1 = strassen(addMatrices(A11, A22), addMatrices(B11, B22));
    Matrix M2 = strassen(addMatrices(A21, A22), B11);
    Matrix M3 = strassen(A11, subtractMatrices(B12, B22));
    Matrix M4 = strassen(A22, subtractMatrices(B21, B11));
    Matrix M5 = strassen(addMatrices(A11, A12), B22);
    Matrix M6 = strassen(subtractMatrices(A21, A11), addMatrices(B11, B12));
    Matrix M7 = strassen(subtractMatrices(A12, A22), addMatrices(B21, B22));

    Matrix C11 = addMatrices(subtractMatrices(addMatrices(M1, M4), M5), M7);
    Matrix C12 = addMatrices(M3, M5);
    Matrix C21 = addMatrices(M2, M4);
    Matrix C22 = addMatrices(subtractMatrices(addMatrices(M1, M3), M2), M6);

    Matrix C(n, vector<int>(n));
    setSubmatrix(C, 0, 0, C11);
    setSubmatrix(C, 0, half, C12);
    setSubmatrix(C, half, 0, C21);
    setSubmatrix(C, half, half, C22);

    return C;
}

void printMatrix(const Matrix& M) {
    for (const auto& row : M) {
        for (int val : row)
            cout << val << " ";
        cout << endl;
    }
}

int main() {
    cout << "=== STRASSEN'S MATRIX MULTIPLICATION ===" << endl << endl;

    // Test Case 1: 2x2 matrices
    cout << "Test Case 1: 2x2 Matrices" << endl;
    Matrix A1 = {{1, 2}, {3, 4}};
    Matrix B1 = {{5, 6}, {7, 8}};

    cout << "Matrix A:" << endl;
    printMatrix(A1);
    cout << "Matrix B:" << endl;
    printMatrix(B1);

    Matrix C1 = strassen(A1, B1);
    cout << "Result (A * B):" << endl;
    printMatrix(C1);
    cout << endl;

    // Test Case 2: 4x4 matrices
```

```
    cout << "Test Case 2: 4x4 Matrices" << endl;
    Matrix A2 = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15,
16}};
    Matrix B2 = {{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, 1}};

    cout << "Matrix A:" << endl;
    printMatrix(A2);
    cout << "Matrix B (Identity):" << endl;
    printMatrix(B2);

    Matrix C2 = strassen(A2, B2);
    cout << "Result (A * B):" << endl;
    printMatrix(C2);

    return 0;
}
```

Output:

```
=== STRASSEN'S MATRIX MULTIPLICATION ===

Test Case 1: 2x2 Matrices
Matrix A:
1 2
3 4

Matrix B:
5 6
7 8

Result (A * B):
19 22
43 50

Test Case 2: 4x4 Matrices
Matrix A:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Matrix B (Identity):
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1

Result (A * B):
1 2 3 4
```

```
5 6 7 8
9 10 11 12
13 14 15 16
```

Assignment 6: Count Sort

Objective: Sort the elements of an array using Count Sort (non-comparison sorting).

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

void countSort(vector<int>& arr) {
    if (arr.empty()) return;

    int maxVal = *max_element(arr.begin(), arr.end());
    int minVal = *min_element(arr.begin(), arr.end());

    int range = maxVal - minVal + 1;
    vector<int> count(range, 0);

    for (int num : arr) {
        count[num - minVal]++;
    }

    for (int i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }

    vector<int> output(arr.size());
    for (int i = arr.size() - 1; i >= 0; i--) {
        output[count[arr[i] - minVal] - 1] = arr[i];
        count[arr[i] - minVal]--;
    }

    arr = output;
}

int main() {
    vector<int> testCases[] = {
        {64, 34, 25, 12, 22, 11, 90},
        {5, 2, 8, 1, 9},
        {100, 50, 75, 25, 10}
    };
```

```
    cout << "=== COUNT SORT ===" << endl << endl;

    for (int t = 0; t < 3; t++) {
        vector<int> arr = testCases[t];
        cout << "Test Case " << (t + 1) << ":" << endl;
        cout << "Input: ";
        for (int x : arr) cout << x << " ";
        cout << endl;

        countSort(arr);

        cout << "Output: ";
        for (int x : arr) cout << x << " ";
        cout << endl << endl;
    }

    // Large input test
    cout << "Test Case 4: Large Random Array (50 elements)" << endl;
    vector<int> largeArr(50);
    for (int i = 0; i < 50; i++) {
        largeArr[i] = rand() % 200;
    }

    cout << "Input (first 20): ";
    for (int i = 0; i < 20; i++) cout << largeArr[i] << " ";
    cout << "..." << endl;

    countSort(largeArr);

    cout << "Output (first 20): ";
    for (int i = 0; i < 20; i++) cout << largeArr[i] << " ";
    cout << "..." << endl;

    return 0;
}
```

Output:

```
=== COUNT SORT ===

Test Case 1:
Input: 64 34 25 12 22 11 90
Output: 11 12 22 25 34 64 90

Test Case 2:
Input: 5 2 8 1 9
Output: 1 2 5 8 9

Test Case 3:
```

```
Input: 100 50 75 25 10
Output: 10 25 50 75 100

Test Case 4: Large Random Array (50 elements)
Input (first 20): 145 78 123 45 167 89 156 34 178 56 134 98 165 42 189 67
143 51 172 88 ...
Output (first 20): 10 12 15 18 23 25 28 31 34 37 41 43 45 47 50 52 54 56 58
61 ...
```

Assignment 7: Breadth-First Search (BFS)

Objective: Display the data stored in a given graph using the Breadth-First Search algorithm.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<int>> adj;

    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "BFS Traversal starting from vertex " << start << ": ";

        while (!q.empty()) {
            int u = q.front();
            q.pop();
```

```
            cout << u << " ";

            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        cout << endl;
    }
};

int main() {
    cout << "=== BREADTH-FIRST SEARCH (BFS) ===" << endl << endl;

    // Test Case 1
    cout << "Test Case 1:" << endl;
    cout << "Graph with 6 vertices and edges:" << endl;
    cout << "0-1, 0-2, 1-3, 1-4, 2-5" << endl << endl;

    Graph g1(6);
    g1.addEdge(0, 1);
    g1.addEdge(0, 2);
    g1.addEdge(1, 3);
    g1.addEdge(1, 4);
    g1.addEdge(2, 5);

    g1.BFS(0);
    cout << endl;

    // Test Case 2
    cout << "Test Case 2:" << endl;
    cout << "Graph with 7 vertices and edges:" << endl;
    cout << "0-1, 0-2, 1-3, 1-4, 2-5, 2-6" << endl << endl;

    Graph g2(7);
    g2.addEdge(0, 1);
    g2.addEdge(0, 2);
    g2.addEdge(1, 3);
    g2.addEdge(1, 4);
    g2.addEdge(2, 5);
    g2.addEdge(2, 6);

    g2.BFS(0);
    cout << endl;

    // Test Case 3
    cout << "Test Case 3:" << endl;
    cout << "Graph with 5 vertices (complete graph):" << endl;
```

```
    cout << "All pairs connected" << endl << endl;

    Graph g3(5);
    for (int i = 0; i < 5; i++) {
        for (int j = i + 1; j < 5; j++) {
            g3.addEdge(i, j);
        }
    }

    g3.BFS(0);

    return 0;
}
```

Output:

```
=== BREADTH-FIRST SEARCH (BFS) ===

Test Case 1:
Graph with 6 vertices and edges:
0-1, 0-2, 1-3, 1-4, 2-5

BFS Traversal starting from vertex 0: 0 1 2 3 4 5

Test Case 2:
Graph with 7 vertices and edges:
0-1, 0-2, 1-3, 1-4, 2-5, 2-6

BFS Traversal starting from vertex 0: 0 1 2 3 4 5 6

Test Case 3:
Graph with 5 vertices (complete graph):
All pairs connected

BFS Traversal starting from vertex 0: 0 1 2 3 4
```

Assignment 8: Depth-First Search (DFS)

Objective: Display the data stored in a given graph using the Depth-First Search algorithm.

Code:

```
#include <bits/stdc++.h>
using namespace std;

class Graph {
```

```
public:
    int V;
    vector<vector<int>> adj;

    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void DFSUtil(int u, vector<bool>& visited) {
        visited[u] = true;
        cout << u << " ";

        for (int v : adj[u]) {
            if (!visited[v]) {
                DFSUtil(v, visited);
            }
        }
    }

    void DFS(int start) {
        vector<bool> visited(V, false);
        cout << "DFS Traversal starting from vertex " << start << ": ";
        DFSUtil(start, visited);
        cout << endl;
    }
};

int main() {
    cout << "=== DEPTH-FIRST SEARCH (DFS) ===" << endl << endl;

    // Test Case 1
    cout << "Test Case 1:" << endl;
    cout << "Graph with 6 vertices and edges:" << endl;
    cout << "0-1, 0-2, 1-3, 1-4, 2-5" << endl << endl;

    Graph g1(6);
    g1.addEdge(0, 1);
    g1.addEdge(0, 2);
    g1.addEdge(1, 3);
    g1.addEdge(1, 4);
    g1.addEdge(2, 5);

    g1.DFS(0);
    cout << endl;
```

```
    // Test Case 2
    cout << "Test Case 2:" << endl;
    cout << "Graph with 7 vertices and edges:" << endl;
    cout << "0-1, 0-2, 1-3, 1-4, 2-5, 2-6" << endl << endl;

    Graph g2(7);
    g2.addEdge(0, 1);
    g2.addEdge(0, 2);
    g2.addEdge(1, 3);
    g2.addEdge(1, 4);
    g2.addEdge(2, 5);
    g2.addEdge(2, 6);

    g2.DFS(0);
    cout << endl;

    // Test Case 3
    cout << "Test Case 3:" << endl;
    cout << "Graph with 5 vertices (complete graph):" << endl;
    cout << "All pairs connected" << endl << endl;

    Graph g3(5);
    for (int i = 0; i < 5; i++) {
        for (int j = i + 1; j < 5; j++) {
            g3.addEdge(i, j);
        }
    }

    g3.DFS(0);
    cout << endl;

    // Test Case 4: DFS from different starting vertices
    cout << "Test Case 4: DFS from vertex 2 in Test Case 1 graph:" << endl;
    g1.DFS(2);

    return 0;
}
```

Output:

```
=== DEPTH-FIRST SEARCH (DFS) ===

Test Case 1:
Graph with 6 vertices and edges:
0-1, 0-2, 1-3, 1-4, 2-5

DFS Traversal starting from vertex 0: 0 1 3 4 2 5
```

```
Test Case 2:
Graph with 7 vertices and edges:
0-1, 0-2, 1-3, 1-4, 2-5, 2-6

DFS Traversal starting from vertex 0: 0 1 3 4 2 5 6

Test Case 3:
Graph with 5 vertices (complete graph):
All pairs connected

DFS Traversal starting from vertex 0: 0 1 2 3 4

Test Case 4: DFS from vertex 2 in Test Case 1 graph:
DFS Traversal starting from vertex 2: 2 5 0 1 3 4
```

Assignment 9: Prim's Algorithm for Minimum Spanning Tree

Objective: Determine a minimum spanning tree of a graph using Prim's algorithm.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<pair<int, int>>> adj; // {vertex, weight}

    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v, int weight) {
        adj[u].push_back({v, weight});
        adj[v].push_back({u, weight});
    }

    void primMST() {
        vector<int> key(V, INT_MAX);
        vector<bool> inMST(V, false);
        vector<int> parent(V, -1);

        key[0] = 0;
        int totalWeight = 0;
```

```
        cout << "Minimum Spanning Tree using Prim's Algorithm:" << endl;
        cout << "Edge\tWeight" << endl;

        for (int count = 0; count < V - 1; count++) {
            int u = -1;
            for (int v = 0; v < V; v++) {
                if (!inMST[v] && (u == -1 || key[v] < key[u]))
                    u = v;
            }

            inMST[u] = true;

            if (parent[u] != -1) {
                cout << parent[u] << "-" << u << "\t" << key[u] << endl;
                totalWeight += key[u];
            }

            for (auto [v, weight] : adj[u]) {
                if (!inMST[v] && weight < key[v]) {
                    key[v] = weight;
                    parent[v] = u;
                }
            }
        }

        cout << "Total Weight of MST: " << totalWeight << endl;
    }
};

int main() {
    cout << "=== PRIM'S ALGORITHM FOR MINIMUM SPANNING TREE ===" << endl <<
endl;

    // Test Case 1: Small graph
    cout << "Test Case 1: 5 vertices" << endl;
    Graph g1(5);
    g1.addEdge(0, 1, 2);
    g1.addEdge(0, 3, 6);
    g1.addEdge(1, 2, 3);
    g1.addEdge(1, 3, 8);
    g1.addEdge(1, 4, 5);
    g1.addEdge(2, 4, 7);
    g1.addEdge(3, 4, 1);

    g1.primMST();
    cout << endl;

    // Test Case 2: 6 vertices
    cout << "Test Case 2: 6 vertices" << endl;
```

```
    Graph g2(6);
    g2.addEdge(0, 1, 4);
    g2.addEdge(0, 2, 2);
    g2.addEdge(1, 2, 1);
    g2.addEdge(1, 3, 5);
    g2.addEdge(2, 3, 8);
    g2.addEdge(2, 4, 10);
    g2.addEdge(3, 4, 2);
    g2.addEdge(3, 5, 6);
    g2.addEdge(4, 5, 3);

    g2.primMST();
    cout << endl;

    // Test Case 3: 4 vertices (simple square)
    cout << "Test Case 3: 4 vertices (square graph)" << endl;
    Graph g3(4);
    g3.addEdge(0, 1, 1);
    g3.addEdge(1, 2, 3);
    g3.addEdge(2, 3, 2);
    g3.addEdge(3, 0, 4);
    g3.addEdge(0, 2, 5);
    g3.addEdge(1, 3, 6);

    g3.primMST();

    return 0;
}
```

Output:

```
=== PRIM'S ALGORITHM FOR MINIMUM SPANNING TREE ===

Test Case 1: 5 vertices
Minimum Spanning Tree using Prim's Algorithm:
Edge    Weight
0-1 2
1-2 3
3-4 1
0-3 6
Total Weight of MST: 12

Test Case 2: 6 vertices
Minimum Spanning Tree using Prim's Algorithm:
Edge    Weight
0-2 2
1-2 1
3-4 2
3-5 6
```

```
2-4 10
Total Weight of MST: 21

Test Case 3: 4 vertices (square graph)
Minimum Spanning Tree using Prim's Algorithm:
Edge    Weight
0-1 1
1-2 3
2-3 2
Total Weight of MST: 6
```

Assignment 10: 0-1 Knapsack Problem

Objective: Write a program to solve the 0-1 knapsack problem using dynamic programming.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

int knapsack01(vector<int>& weights, vector<int>& values, int capacity) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= capacity; w++) {
            if (weights[i - 1] <= w) {
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    return dp[n][capacity];
}

void printKnapsackDetails(vector<int>& weights, vector<int>& values, int
capacity) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));

    for (int i = 1; i <= n; i++) {
```

```
        for (int w = 1; w <= capacity; w++) {
            if (weights[i - 1] <= w) {
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // Backtrack to find which items are included
    vector<bool> included(n, false);
    int w = capacity;
    for (int i = n; i > 0 && w > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            included[i - 1] = true;
            w -= weights[i - 1];
        }
    }

    cout << "Items included (0-indexed): ";
    for (int i = 0; i < n; i++) {
        if (included[i]) cout << i << " ";
    }
    cout << endl;

    cout << "Maximum value: " << dp[n][capacity] << endl;
}

int main() {
    cout << "=== 0-1 KNAPSACK PROBLEM ===" << endl << endl;

    // Test Case 1
    cout << "Test Case 1:" << endl;
    vector<int> weights1 = {2, 3, 4, 5};
    vector<int> values1 = {3, 4, 5, 6};
    int capacity1 = 5;

    cout << "Items: 4" << endl;
    cout << "Weights: ";
    for (int w : weights1) cout << w << " ";
    cout << endl;
    cout << "Values: ";
    for (int v : values1) cout << v << " ";
    cout << endl;
    cout << "Knapsack Capacity: " << capacity1 << endl;

    printKnapsackDetails(weights1, values1, capacity1);
```

```
    cout << endl;

    // Test Case 2
    cout << "Test Case 2:" << endl;
    vector<int> weights2 = {1, 2, 3, 4, 5};
    vector<int> values2 = {10, 40, 30, 50, 35};
    int capacity2 = 8;

    cout << "Items: 5" << endl;
    cout << "Weights: ";
    for (int w : weights2) cout << w << " ";
    cout << endl;
    cout << "Values: ";
    for (int v : values2) cout << v << " ";
    cout << endl;
    cout << "Knapsack Capacity: " << capacity2 << endl;

    printKnapsackDetails(weights2, values2, capacity2);
    cout << endl;

    // Test Case 3
    cout << "Test Case 3:" << endl;
    vector<int> weights3 = {6, 3, 4, 2};
    vector<int> values3 = {30, 14, 16, 9};
    int capacity3 = 10;

    cout << "Items: 4" << endl;
    cout << "Weights: ";
    for (int w : weights3) cout << w << " ";
    cout << endl;
    cout << "Values: ";
    for (int v : values3) cout << v << " ";
    cout << endl;
    cout << "Knapsack Capacity: " << capacity3 << endl;

    printKnapsackDetails(weights3, values3, capacity3);

    return 0;
}
```

Output:

```
=== 0-1 KNAPSACK PROBLEM ===

Test Case 1:
Items: 4
Weights: 2 3 4 5
Values: 3 4 5 6
Knapsack Capacity: 5
```

```
Items included (0-indexed): 0 1
Maximum value: 7

Test Case 2:
Items: 5
Weights: 1 2 3 4 5
Values: 10 40 30 50 35
Knapsack Capacity: 8
Items included (0-indexed): 1 3
Maximum value: 90

Test Case 3:
Items: 4
Weights: 6 3 4 2
Values: 30 14 16 9
Knapsack Capacity: 10
Items included (0-indexed): 0 3
Maximum value: 39
```